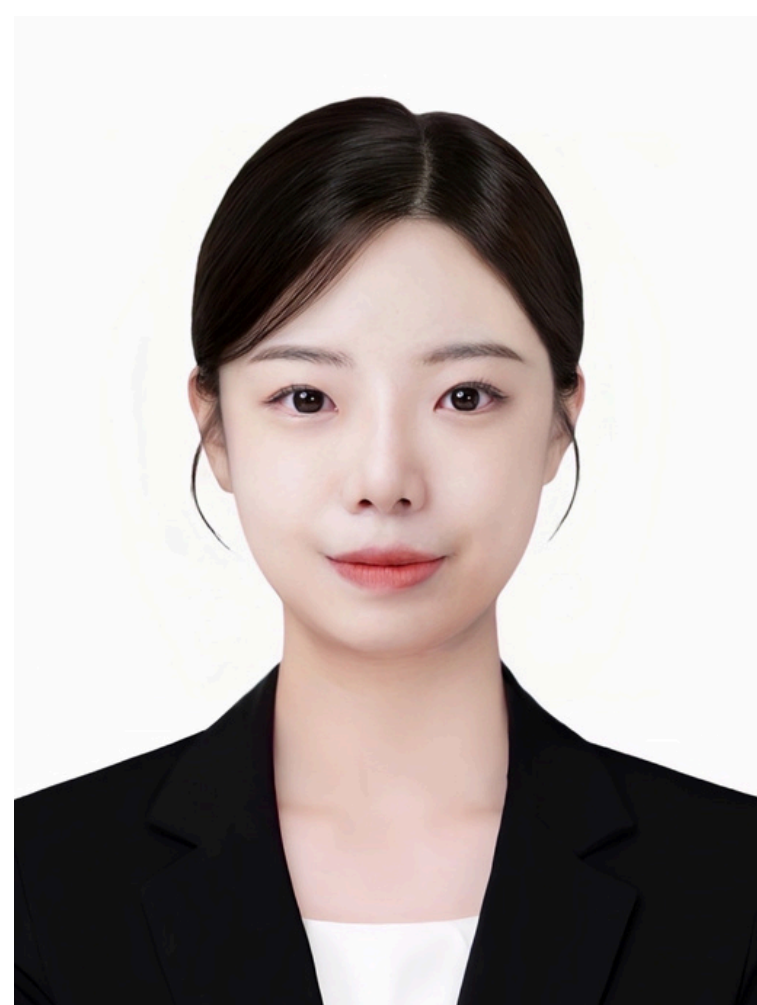




윤경은

검색 랭킹 개선 · 사용자 의도 기반 라우팅 · ML 서빙 안정화 · 데이터 기반 가설 검증

문제를 더 잘 정의해서, ML이 실제로 먹히게 만듭니다

응용통계학(GARCH·K-means·PCA) 기반의 정량 분석 경험 위에, 검색 품질 평가·의도 분류 라우팅·서빙 안정화·실험 해석 경험을 쌓아 왔습니다. 당근 ML 직무에서도 중요할 **검색 품질 평가, 사용자 의도 해석, 비용 통제, 서빙 안정성**을 측정 가능한 문제로 바꾸고 평가 기준부터 설계하는 방식에 익숙합니다.

"감"이 아닌 데이터로 의사결정합니다.

최근 프로젝트들에서 팀이 "LLM을 바꾸자"고 할 때 RAGAS로 병목을 검색 단계로 다시 정의했고, LoRA 3회 실험에서는 성능이 -3.2%p 하락하자 약점 19건 타겟팅으로 전략을 수정했습니다. 정답이 하나로 정해지지 않은 문제에서도 지표와 제약을 함께 보며 더 나은 균형점을 찾습니다.

0.83 → 0.97

검색 정밀도 +14%p

RAGAS로 병목을 찾은 뒤 검색 랭킹을 재설계해 관련 없는 문서 노출을 줄이고 필요한 문서가 먼저 보이도록 개선

37% → 85%

규정 판단 정확도

328건 사내 규정 평가셋 기준, vLLM 서빙·LoRA 3회 실험·Guardrail 설계로 고신뢰 오답을 줄이며 정확도와 안정성을 함께 개선

~\$0.02 → ~\$0.003

추정 요청 비용 85% 절감

GPT-4o-mini 토큰 단가 기준, 의도 분류 라우팅으로 불필요한 LLM 호출을 줄여 비용과 응답 흐름을 함께 최적화

About

Why Me For Daangn ML

검색 품질 개선

사용자 의도 해석

서빙 안정성

비용 최적화

문제를 다시 정의하고, 검색·추천 흐름을 단계별로 나눠 보는 방식에 익숙합니다. 관련 문서가 먼저 보이게 랭킹을 손보고, 불필요한 호출은 라우팅으로 줄이고, 잘못된 응답은 서빙 레이어에서 한 번 더 걸러내는 방식으로 실제 서비스 품질을 개선해 왔습니다.

Email yge0307@gmail.com

GitHub github.com/ykgstar37-lab

Portfolio yge-portfolio.vercel.app

Education 가천대학교 응용통계학과 (2020.03 ~ 2026.08)

SK네트웍스 Family AI Camp (2025.09 ~ 2026.03)

Skills

★ AGENT FRAMEWORK

LangGraph

LangChain

MCP

★ LLM SERVING

vLLM

LoRA

OpenAI API

★ RAG / MEMORY

ChromaDB

Qdrant

HyDE

BM25

Reranker

★ EVAL / QUALITY

RAGAS

Guardrail

Confidence Cal.

★ BACKEND

FastAPI

Django

SSE

PostgreSQL

★ INFRA

Docker

AWS

Nginx

Git

CORE COMPETENCY

01 ML 서빙 + 파인튜닝 - 모델 성능과 운영 안정성을 함께 봅니다

정확도를 올려도 JSON 파싱이 깨지면 서비스는 장애입니다. 모델 성능 실험과 서빙 안정성 설계를 분리하지 않고, 실험 결과가 기대와 다를 때 가설을 고집하지 않고 학습 전략을 바로 수정합니다. 데이터를 늘렸더니 오히려 성능이 떨어진 경험에서, "양보다 구성"이라는 원칙을 직접 체득했습니다. → Workflow Agent 참고

02 검색 랭킹 설계 - "관련 결과가 먼저 보이는가"를 기준으로

RAG 검색 파이프라인에서 쿼리 이해 → 후보 생성 → 리랭킹의 단계별 병목을 지표로 분리 측정하고 구조를 다시 설계합니다. learning-to-rank 모델을 직접 학습시킨 경험은 아니지만, 평가 기준을 먼저 세우고 단계별로 어디가 병목인지 데이터로 분리하는 사고는 검색·추천 품질 문제를 이해하고 배우는 데 이어질 수 있습니다.→ PyMate 참고

03 데이터 기반 의사결정 - 감이 아닌 지표로 방향을 바꿉니다

응용통계학 전공(가설 검정, 모형 선택, 군집분석)을 ML 실험 설계에 적용합니다. 팀이 "LLM을 바꾸자"고 할 때 RAGAS 지표를 제안해 검색·생성을 분리 측정하고 병목을 재정의했고, 파인튜닝 실험에서는 평가셋 규모에 따른 신뢰구간을 고려해 판단합니다. 오프라인 지표를 해석하고 다음 실험 또는 제품 개선 액션으로 연결하는 과정을 강점으로 키워 왔습니다.

PyMate

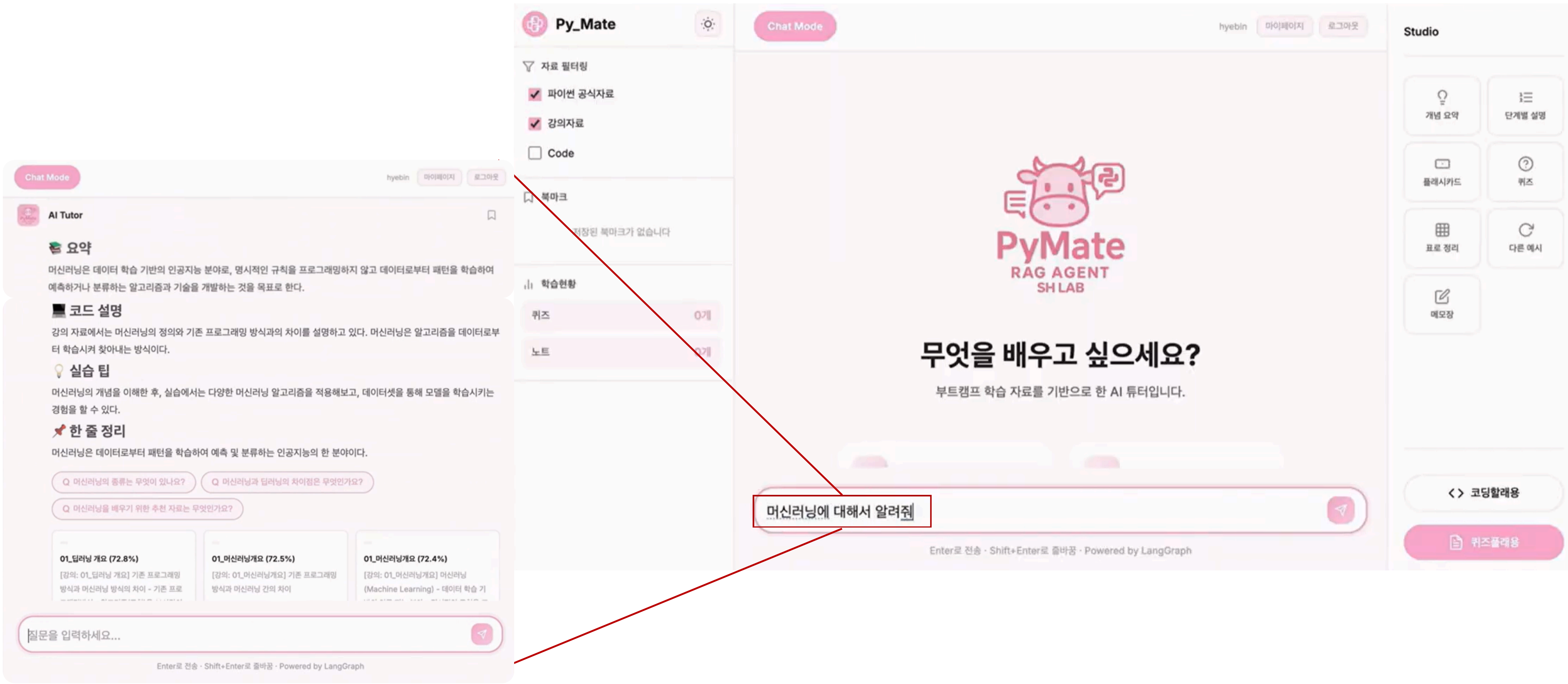
GitHub

RAGAS 기반 RAG 병목 진단 + Flask → Django 프로덕션 전환

"LLM이 문제"라는 가정을 RAGAS로 반증하고, RAG 검색 구조를 다시 설계해 Precision 0.83→0.97로 개선

ROLE FIT

RAG 검색 품질 평가, 하이브리드 검색 파이프라인 설계, 병목 분리 측정 경험을 가장 가깝게 보여주는 프로젝트입니다. 모델 교체보다 먼저 지표를 분해해 병목을 찾고, 검색 구조를 다시 설계해 관련 문서가 먼저 보이도록 만들었습니다.



개요

AI Camp에서 부트캠프 학생들이 강의 내용을 스스로 검증할 도구가 없었습니다. 기존 RAG 챗봇을 만들었지만 "답변이 부정확하다"는 피드백이 반복되었고, 원인이 LLM인지 검색인지 구분할 수 없는 상태였습니다.

Python 공식 문서 + 강의 자료 기반 RAG 학습 튜터 챗봇. 처음에는 "더 좋은 LLM으로 바꾸면 해결될 것"처럼 보였지만, RAGAS 4개 지표로 검색·생성을 분리 측정해 진짜 병목이 embedding 품질이라는 점을 확인했습니다. 이후 검색 랭킹 구조를 다시 설계하고 Flask → Django 프로덕션 전환 + AWS 배포까지 이어졌습니다.

역할

기여도 팀 프로세스 기준, 검색 품질 진단·랭킹 구조·프로덕션 전환을 맡은 범위입니다.

70%

(팀 4명, 6주): Qdrant 벡터 DB 설계(문서 500+건 임베딩), 임베딩 차원 최적화(768D→3,072D, 4배 확장), Flask→Django 마이그레이션, AWS EC2(Nginx+Gunicorn) 프로덕션 배포

Skills

Django

Qdrant

RAGAS

OpenAI Embedding (3072D)

LangChain

Nginx

Gunicorn

AWS EC2

COMPARE

Flask → Django 프로덕션 전환 + AWS 배포

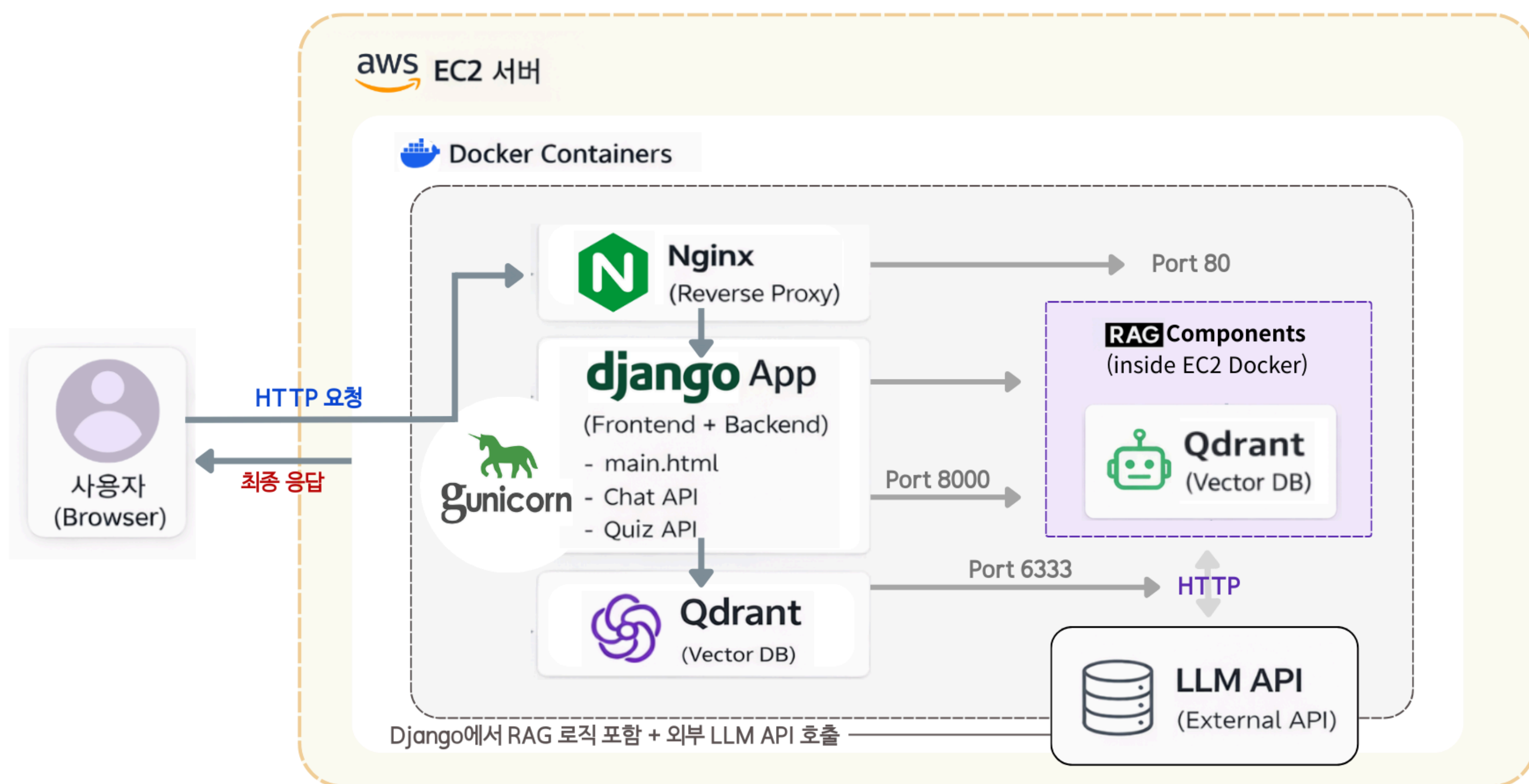
AS-IS

- MVP 한계:** Flask MVP에는 ORM 마이그레이션, 정적 파일 서빙, 관리자 페이지 같은 운영 기능이 부족했습니다.
- 배포 불안정:** AWS EC2에서 Nginx → Gunicorn → Flask 연결의 소켓 바인딩 문제로 502 에러가 반복됐습니다.
- 서비스 리스크:** 기능 실험은 가능했지만, 운영 가능한 형태로 확장하기엔 인프라 표준화가 부족했습니다.

TO-BE

- 인프라 표준화:** Django의 ORM migration, admin, collectstatic으로 운영 구조를 정리했습니다.
- 배포 안정화:** Nginx → Gunicorn → Django 소켓 바인딩을 직접 맞춰 502 에러를 해결했습니다.
- 실서비스 전환:** Reranker를 cross-encoder로 경량화해 레이턴시를 약 1초 단축했고, 실험용 챗봇을 배포 가능한 형태로 바꿨습니다.

TO-BE FLOW



위 구조는 "LLM 교체" 대신 검색 병목을 분리 진단한 뒤 실제로 적용한 하이브리드 검색·랭킹 파이프라인과 Django/AWS 운영 구조를 한 번에 보여줍니다.

COMPARE

RAG 품질 병목 진단 – RAGAS 지표로 검색·생성 분리 측정

AS-IS

문제 정의 부재: "답변이 부정확하다"는 피드백만 반복되고, 검색 문제인지 생성 문제인지 분리 측정이 불가능했습니다.

잘못된 대응 방향: 병목 확인 없이 바로 LLM 교체로 가자는 의견이 나와, 비용만 늘고 원인을 놓칠 위험이 있었습니다.

제품 리스크: 관련 없는 문서가 먼저 노출되어 첫 답변 성공률이 흔들리고, 재질문이 늘어날 가능성이 컸습니다.

TO-BE

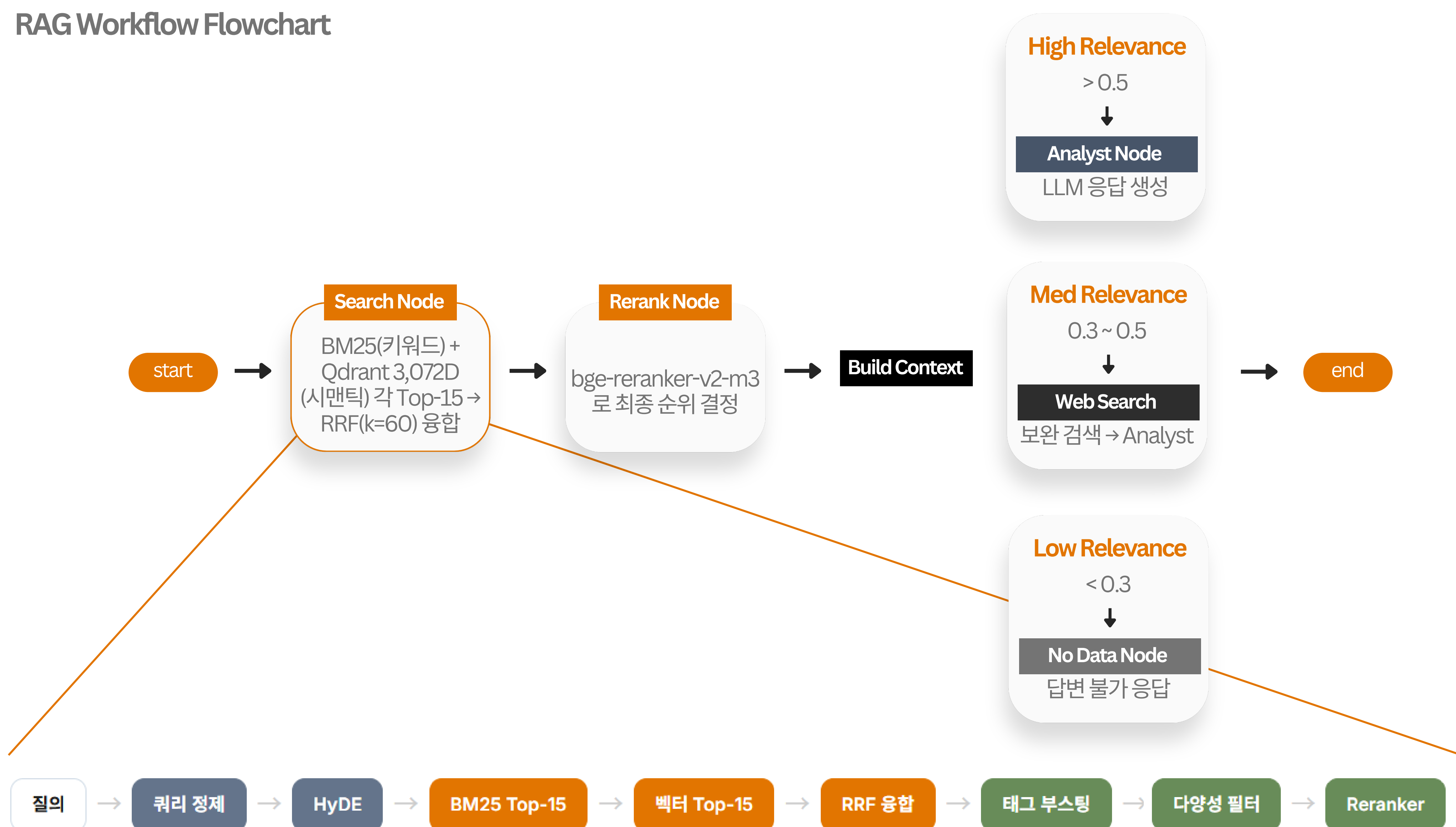
병목 분리: RAGAS 4개 지표로 검색·생성을 분리 측정해, Faithfulness는 높고 Precision이 낮다는 근거로 embedding 병목을 특정했습니다.

해결 구조: 768D→3,072D 교체, HyDE, BM25+RRF를 조합해 짧은 질문과 긴 문서 사이의 vocabulary mismatch를 줄였습니다.

결과 해석: Precision 0.83→0.97 (+14.4%p). 관련 없는 문서 노출을 줄여 첫 답변 성공률과 재질문 감소의 선행지표로 해석했습니다.

TO-BE FLOW

RAG Workflow Flowchart



Search Node 내부 – 9-Stage Hybrid Pipeline

쿼리 정제 + HyDE

형태소 분석 + 동의어 확장 + 구어→문어 변환. HyDE로 가설 답변 문서를 생성하여 검색 품질 향상.

BM25 + 벡터 + RRF

BM25(키워드) + Qdrant 3,072D(시맨틱) 각 Top-15를 RRF(k=60)로 융합. 키워드·의미 검색의 상호 보완.

Reranker + RAGAS 진단

bge-reranker-v2-m3로 최종 순위 결정. RAGAS 4개 지표로 각 단계별 품질을 분리 측정하여 병목 특정.

쿼리 정제 → HyDE → BM25/Qdrant 후보 생성 → RRF 융합 → Reranker 최종 정렬

관련 문서 상위 노출

AS-IS  0.83

TO-BE  0.97

관련 문서가 먼저 보이도록 랭킹 구조를 재설계했습니다. (+14%p)

검색 커버리지 확대

AS-IS  0.70

TO-BE  0.79

Recall도 함께 올려 검색 커버리지를 넓히고 누락을 줄였습니다. (+9%p)

RAG 병목 재정의

embedding

RAGAS로 검색·생성 분리 측정 후 embedding 병목 확인했습니다.

회고

"LLM 교체"가 아니라 **병목을 다시 정의하는 것**이 핵심이었습니다. RAGAS로 검색·생성을 분리 측정해 embedding 병목을 확인했고, HyDE도 단순 query expansion보다 문서 수준 의미를 더 보강하는 retrieval 최적화 관점에서 선택했습니다.

현재 500건 규모에서는 가능했지만, 더 큰 문서·트래픽 환경에서는 ANN(HNSW) + pre-filter 기반 2-stage 구조와 더 큰 평가셋 검증이 필요합니다.

WorkFlow Agent

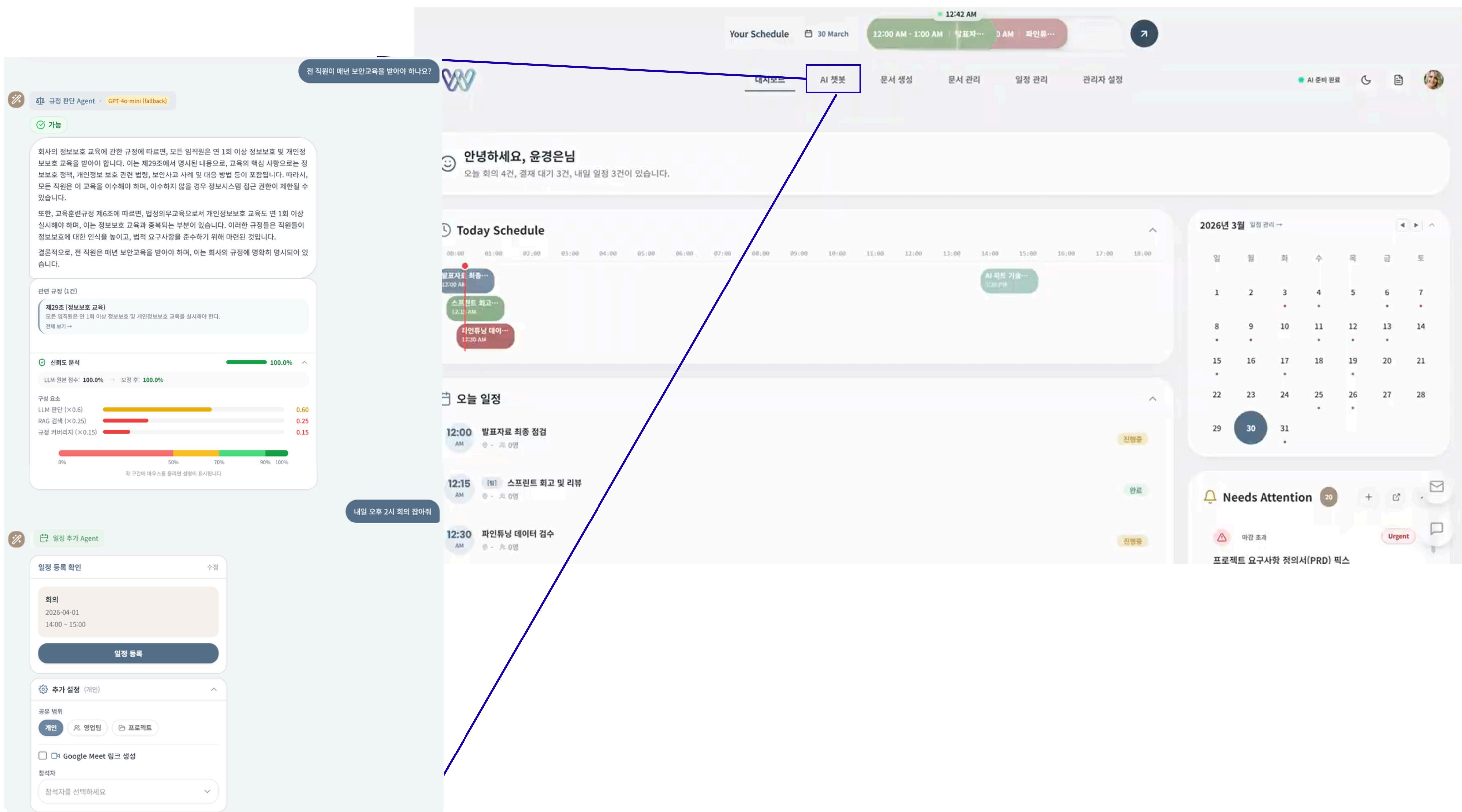
GitHub

vLLM 기반 sLLM 서빙 + 4중 Guardrail + Confidence 보정 시스템

사내 규정 질의에 대한 LLM 판단의 오답·환각·과신을 시스템적으로 차단하는 품질 검증 체계 설계

ROLE FIT

ML 서빙 안정화, 모델 출력 검증, 데이터셋 반복 실험 역량이 드러납니다. 정확도만 올리는 데서 멈추지 않고, 잘못된 응답이 서비스 레이어를 통과하지 않도록 검증 구조까지 설계했습니다.



개요

사내 업무 자동화 에이전트를 개발하면서, GPT API에 사내 데이터를 보낼 수 없는 보안 제약(온프레미스 서빙 필요)과 LLM이 "확신하면서 틀리는" 문제를 동시에 해결해야 했습니다.

사내 규정 판단·문서 처리·일정 관리·복합 요청 분해를 4개 전문 Agent가 처리하는 업무 자동화 시스템. vLLM 기반 온프레미스 sLLM(Kanana-1.5-8B) 서빙으로 GPT 의존을 제거하고, 4중 Guardrail + Confidence 보정으로 Agent의 오답·환각·과신을 시스템적으로 차단합니다.

역할

기여도

팀 프로세스 기준, 검증 체계·서빙 안정화·실험 루프를 맡은 범위입니다.

65%

(팀 4명, 8주) – Judgment Agent 품질 보장 4중 Guardrail 설계, 5-factor Confidence 보정, LoRA 파인튜닝 3회 반복(학습 데이터 3,468건 + 평가 328건), 9단계 하이브리드 RAG 파이프라인(HyDE+BM25+RRF+Reranker)

Skills

vLLM

LoRA

FastAPI

RAG

Qdrant

LangChain

Docker

COMPARE

Agent 출력 검증 - 4중 Guardrail + Confidence 보정

AS-IS

위험한 실패: "인턴에게 AWS 접근 권한을 줘도 되나요?"
질의에 Base 모델이 오답을 내면서도 높은 confidence를 반환했습니다.

환각 문제: 존재하지 않는 조항 번호를 근거처럼 제시해,
잘못된 응답이 더 신뢰롭게 보이는 상황이
발생했습니다.

기본 성능: 규정 판단 정확도가 37.2%에 머물러 서비스
레이어에서 바로 쓰기 어려웠습니다.

TO-BE

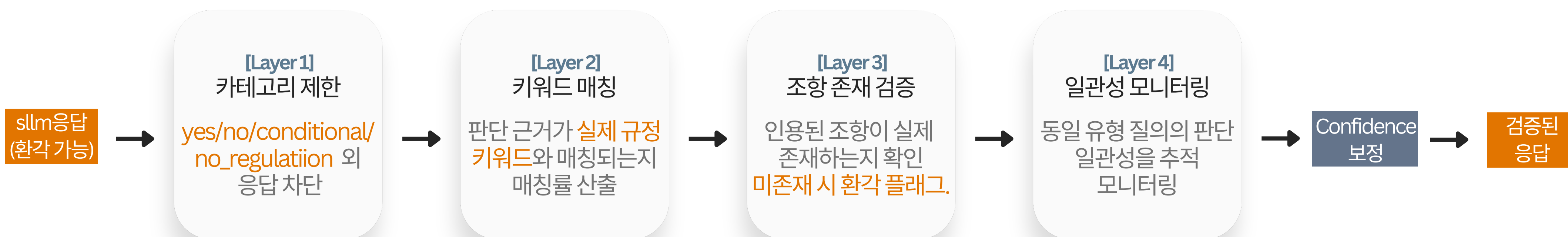
차단 구조: 4중 Guardrail로 키워드 매칭, 조항 존재,
카테고리 제한, 500건 FIFO 일관성 모니터링을
분리 적용했습니다.

Confidence 보정: RAG 품질이 0.2 미만이면 confidence를
max 0.4로 제한해, 고신뢰 응답이 실행 단계로
넘어가지 않도록 설계했습니다.

결과: 37%→85%. 보안·감사 가능성·낮은 latency가 중
요한 환경이라 전수 ML 검증보다 규칙 기반으로
명확한 실패를 먼저 막았습니다.

TO-BE FLOW

4-Layer Guardrail Pipeline



같은 쿼리에서의 Before/After

사용자 질문 → [RAG 컨텍스트]: 정보시스템 제17조, 접근통제 제12조

인턴에게 AWS 접근 권한을 줘도 되나요?

BEFORE - BASE(37.2%)

"result": "yes" 오답 (정답: conditional)
"confidence": 0.92 근거 없이 과신
"reasoning": "접근 권한 부여가 가능합니다"
"regulations": [{"article": "제12조"}] 존재하지 않는 조항
"conditions": -
"alternatives": -
누락

AFTER - LoRA v1 (85.4%)

"result": "conditional" 정답
"confidence": 0.78 불확실성 반영
"reasoning": "수습기간 80% 이상 근무 후 제한적 접근 가능"
"regulations": [{"article": "정보보안규정 제25조"}, {"article": "인사규정 제8조"}] 실존 조항
"conditions": "보안 교육 이수 + 부서장 승인"
"alternatives": ["테스트 환경 한정 접근"]

위 구조는 AS-IS의 오답·환각·과신을 TO-BE의 검증 가능한 서비스 레이어로 바꾸기 위해 설계한 멀티 에이전트 + Guardrail 흐름입니다.

COMPARE

LoRA 파인튜닝 - 데이터 양 vs 품질 실험

AS-IS

초기 가설: LoRA v2에서 "데이터를 많이 넣으면 좋아진다"는 가정으로 98건을 추가했습니다.

실험 결과: 성능이 오히려 -3.2%p 하락해, 양적 확대가 바로 개선으로 이어지지 않았습니다.

원인: 재량적 표현이 섞인 라벨 오염이 발견되어 데이터 품질 자체가 병목임을 확인했습니다.

TO-BE

정밀 타겟팅: LoRA v3에서 약점 19건만 다시 설계해 재량 표현 14건, 경계 케이스 5건을 집중 보완했습니다.

판단 전환: 데이터 양 확대 대신 데이터셋 구성 자체를 바꾸며, 틀린 가설을 바로 수정하는 방향으로 실험 전략을 바꿨습니다.

트레이드오프: vLLM은 보안·온프레미스 요구 때문에, LoRA는 full fine-tuning 대비 메모리 효율과 실험 속도 때문에 선택했습니다.

TO-BE FLOW

3단계 실험 여정

“RAG를 학습에 넣으면 좋을까?” → 실험으로 답을 찾다

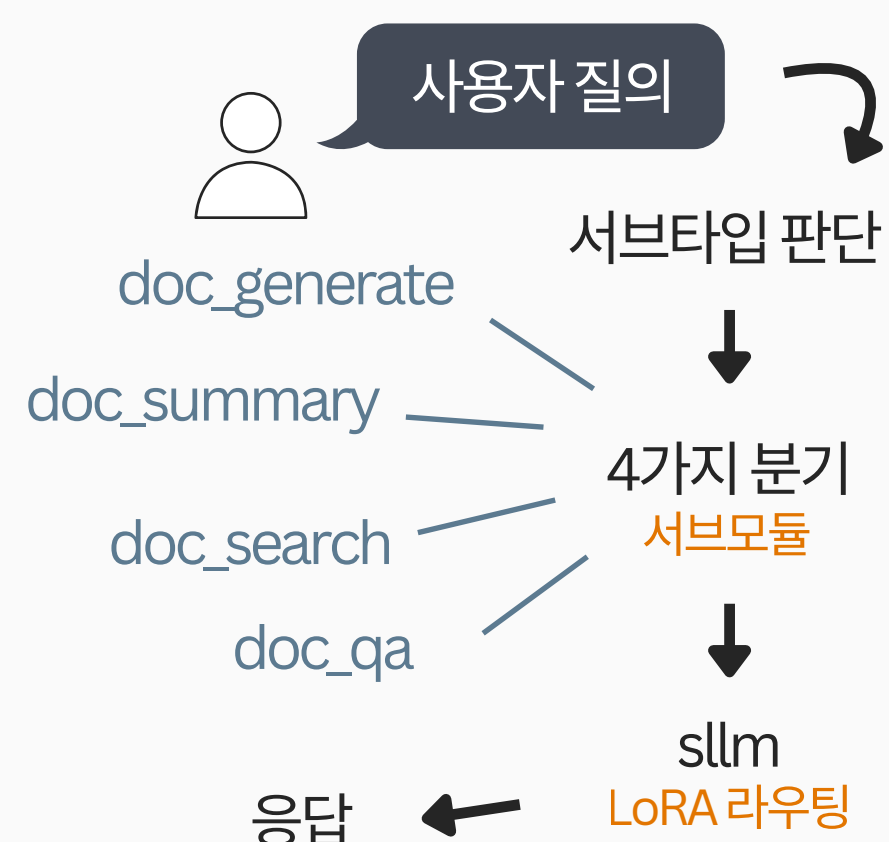


v2에서 RAG 노이즈로 -9.8%p 급락 → “학습은 하드코딩(깨끗한 신호), 서빙만 RAG(실시간 검색)” 전략 확정

Multi-Agent Architecture

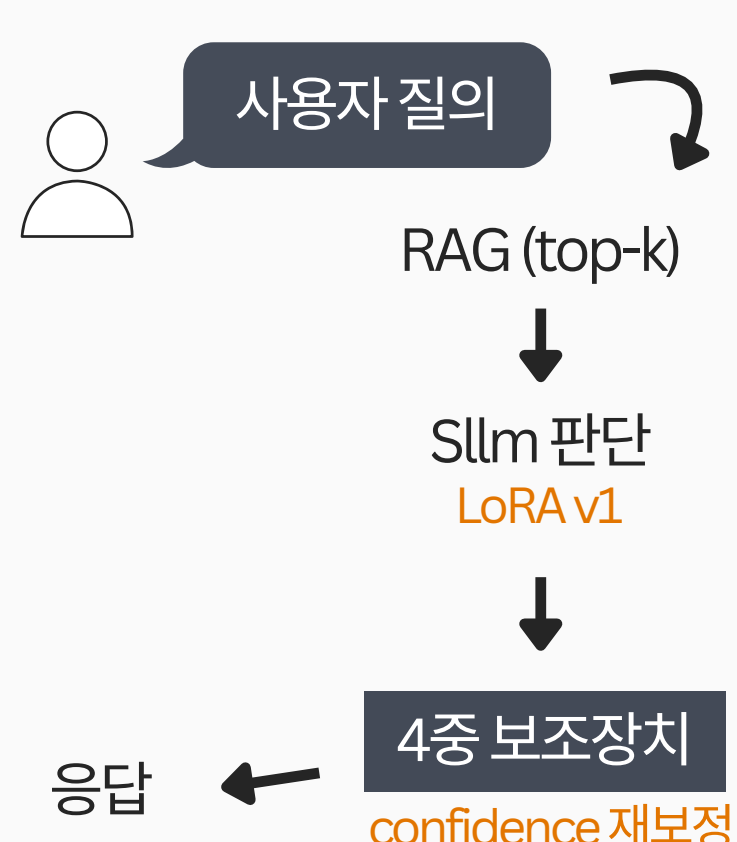
문서 AGENT

문서
생성/요약/검색/QA
4가지 오퍼레이션



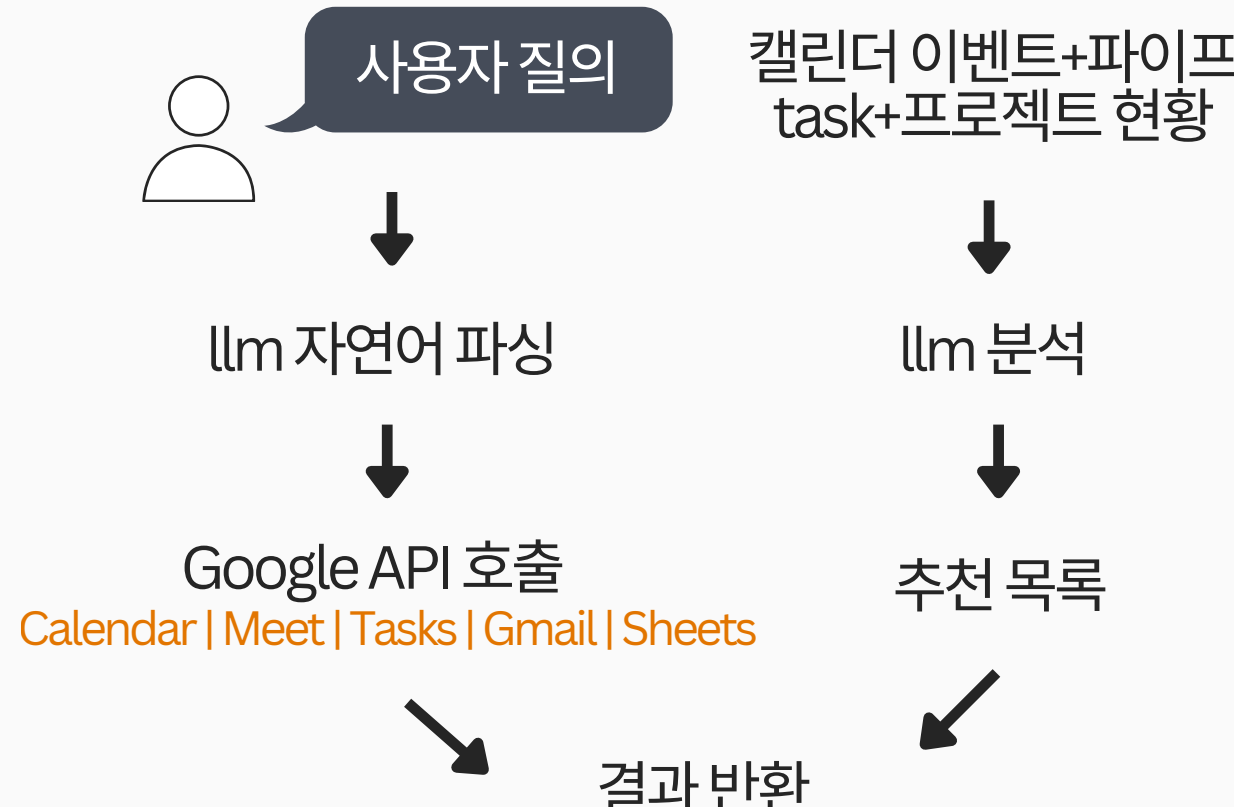
판단 AGENT

사내규정 기반
yes/no/conditional 판단
+ 근거 조항 + 대안



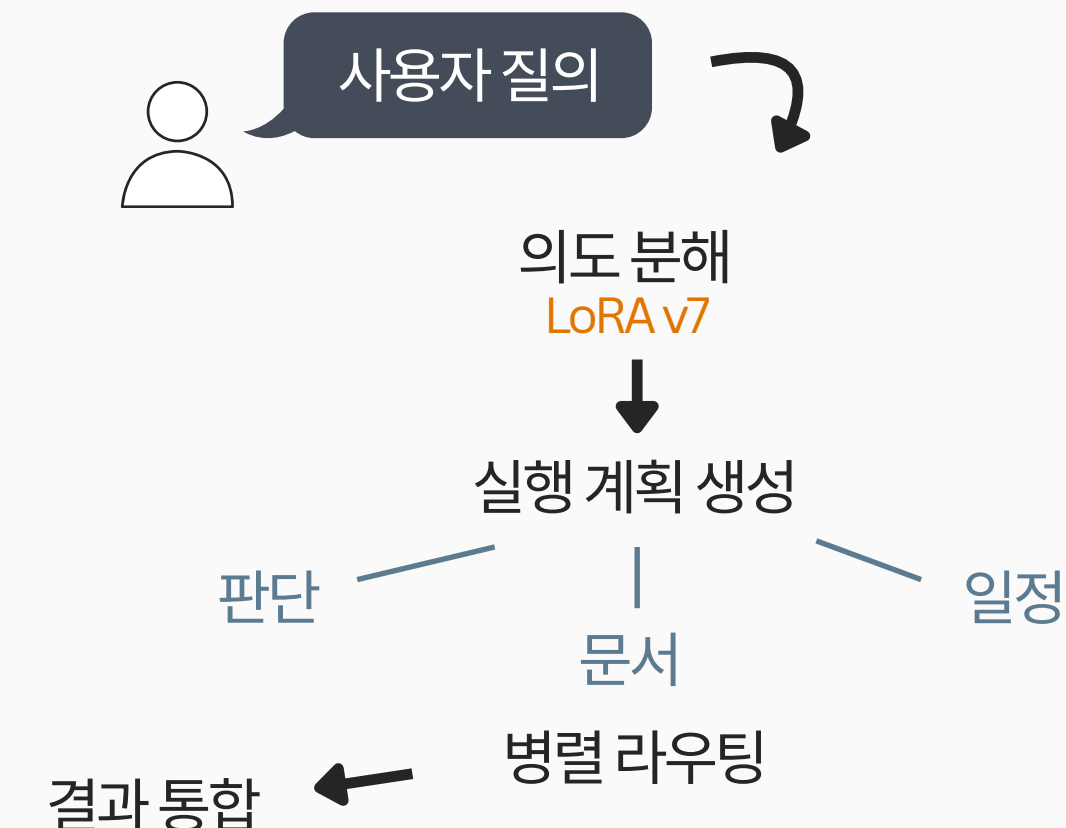
일정 AGENT

자연어
→ 일정 등록/조회
+ Google 5종 연동



Planner AGENT

복합 요청
→ 단계별 계획 분해
+ 병렬 실행



규정 판단 정확도

AS-IS  37%

TO-BE  85%

328건 사내 규정 평가셋 기준으로 정확도와 안정성을 함께 끌어올렸습니다. (+48%p)

과신 응답 보정

AS-IS  0.92

TO-BE  0.78

과신 confidence를 보정해, 틀리면서 확신하는 응답을 줄였습니다. (-0.14)

데이터 정밀 보완

AS-IS TO-BE
98건 → **19건**

데이터 양 확대보다 약점 19건 정밀 보완이 더 효과적이라는 점을 실험으로 확인했습니다. (-79건, 약 81% 감소)

회고

핵심은 실험 → 가설 반증 → 데이터셋 재설계 루프였습니다. LoRA v2의 -3.2%p 하락을 보고 양적 확대보다 데이터 품질과 서비스 레이어 검증이 더 중요하다는 판단으로 방향을 바꿨습니다.

328건 평가셋은 프로토타입 검증엔 충분했지만, 프로덕션 수준에서는 도메인별 1,000건+ 평가셋과 CI 기반 지속 평가가 필요합니다.

Seoul Culture Map

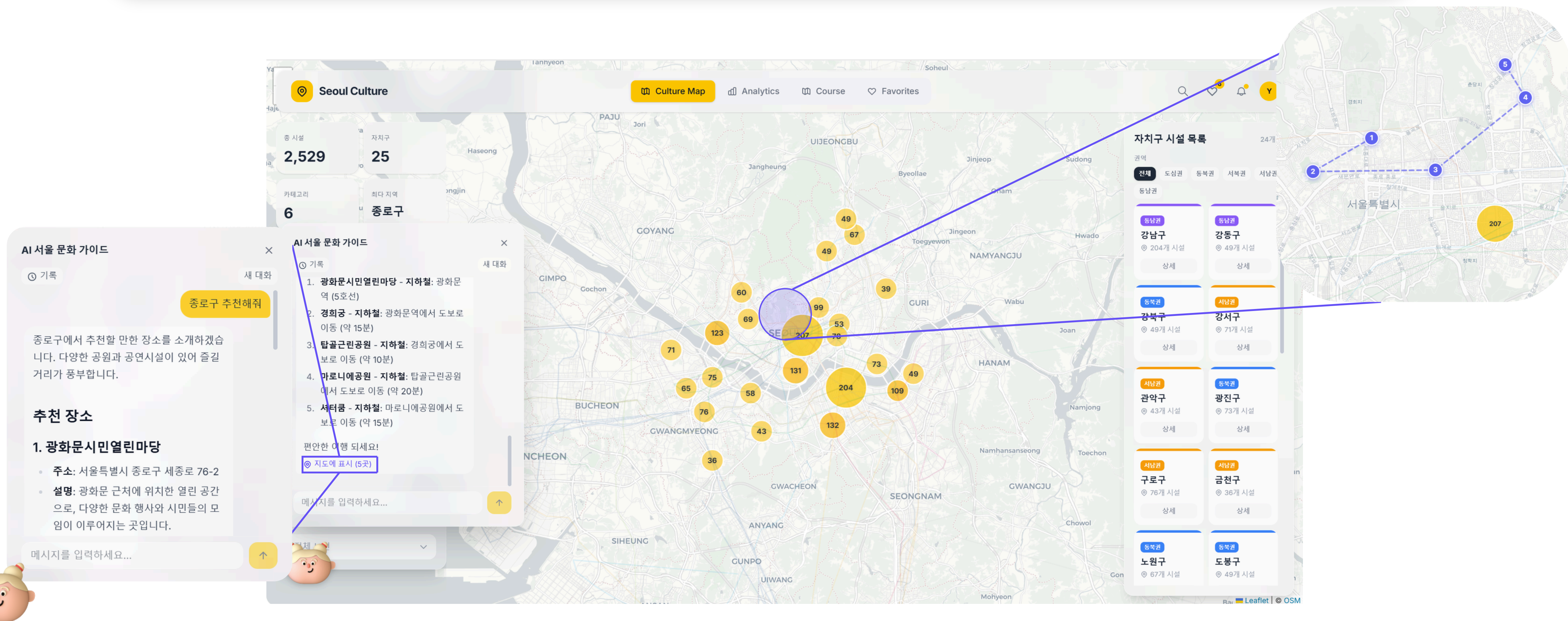
[GitHub](#)

LangGraph 3-node AI Agent + ChromaDB RAG + SSE 스트리밍 챗봇

모든 요청에 LLM을 호출하던 비효율을 의도 분류 라우팅으로 줄여 GPT-4o-mini 토큰 단가 기준 추정 요청당 비용을 ~\$0.02→\$0.003로 절감

ROLE FIT

사용자 의도 해석, 로컬 맥락 기반 탐색, 비용 효율 라우팅 관점이 드러납니다. 모든 요청에 LLM을 쓰기보다 무엇을 먼저 분류하고 어디에만 생성 모델을 써야 하는지 판단하는 흐름을 설계했습니다.



개요

2023년 학술제에서 서울시 25개 자치구 관광시설 데이터를 R로 군집분석하여 2등상을 수상했지만, 결과물이 PDF 보고서로만 남아 실제 사용자가 활용할 수 없었습니다. "분석 결과를 누구나 탐색하고 질문할 수 있는 서비스로 만들 수 있을까?"라는 문제의식에서 출발했습니다.

서울시 2,500+ 문화시설을 인터랙티브 지도에서 탐색하고, LangGraph 기반 Agentic RAG 챗봇으로 자연어 질의(검색·추천·일상대화)를 처리하는 대화형 AI 안내 서비스. 의도 분류로 불필요한 LLM 호출을 제거하고, SSE 스트리밍으로 실시간 응답을 전달합니다.

역할

기여도

개인 프로젝트로 문제 정의부터 라우팅·검색·프론트 구현까지 전체를 담당했습니다.

100%

전체 설계 및 개발 (개인, 4주) LangGraph 3-node Agent 파이프라인, ChromaDB RAG 검색 (2,500+ 시설), 15개 REST/SSE API 설계, React 19 프론트엔드

Skills

LangGraph

ChromaDB

FastAPI

SSE

SQLite

React 19

Leaflet

scikit-learn

OpenAI GPT-4o-mini

COMPARE

Agent 파이프라인 설계 - 의도 분류 + 조건분기

AS-IS

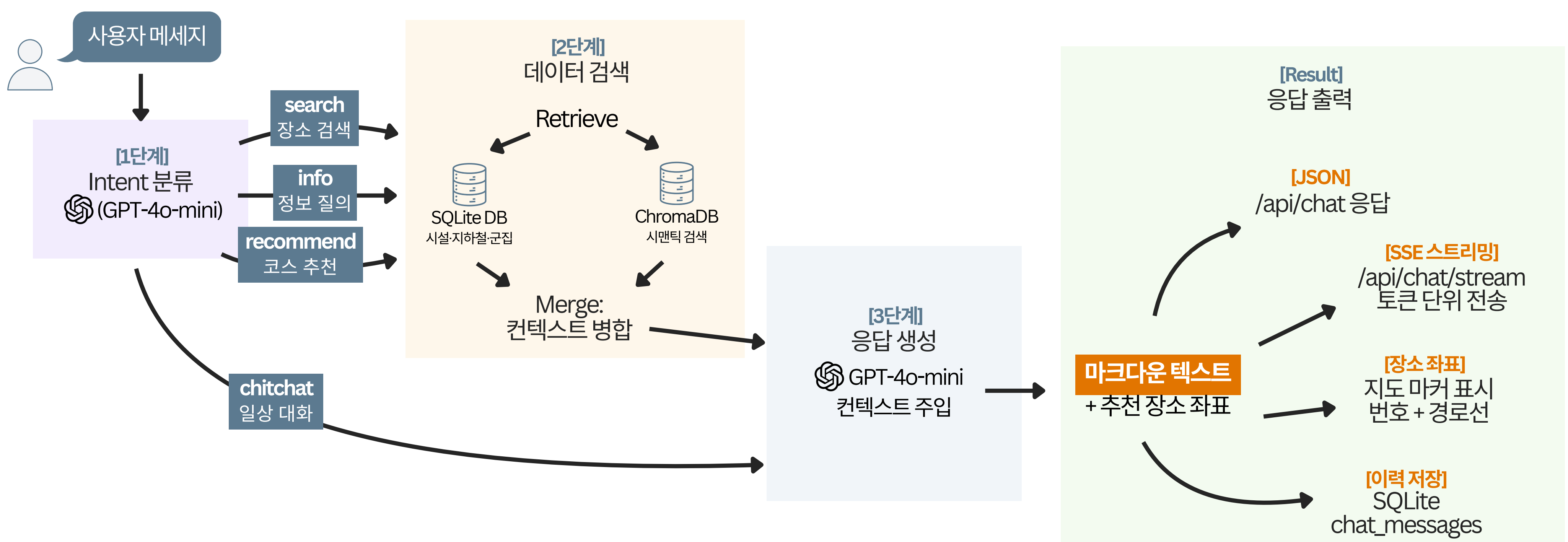
- 과한 단일 처리:** 2,500+ 시설 질의를 모두 단일 LLM 호출로 처리해, 모든 요청에 전체 컨텍스트를 주입하고 있었습니다.
- 비용 비효율:** 단순 인사와 실제 검색 질의가 같은 파이프라인을 타면서 토큰 비용이 불필요하게 커졌습니다.
- 품질 제어 한계:** 의도 분류, 검색, 생성이 섞여 있어 어느 단계가 실패했는지 제어하기 어려웠습니다.

TO-BE

- 노드 분리:** LangGraph 3-node 구조로 의도 분류, 검색, 응답 생성을 분리하고 각 노드를 독립적으로 테스트 가능하게 만들었습니다.
- 불필요한 호출 제거:** 검색이 필요 없는 요청에는 DB/벡터 쿼리 자체를 건너뛰고, 로컬 임베딩으로 외부 API 비용도 줄였습니다.
- 결과:** GPT-4o-mini 토큰 단가 기준 추정 요청당 비용을 ~\$0.02/req → ~\$0.003/req로 줄이며, "언제 검색하지 않을지"가 핵심 최적화 포인트임을 확인했습니다.

TO-BE FLOW

의도 분류 기반 3-Node 응답 파이프라인



Node 1 - Intent 분류

GPT-4o-mini로 3종 의도(검색·추천·일상 대화) 분류. 일상대화는 검색 노드를 스킵하여 불필요한 DB/벡터 비용 제거.

Node 2 - Retrieve

LLM 없이 SQL(필터) + ChromaDB(시맨틱) 병렬 검색. 로컬 임베딩(all-MiniLM-L6-v2)으로 API 비용 \$0.

Node 3 - Generate

검색 결과 + 세션 메모리(10턴)를 컨텍스트로 주입. SSE 토큰 스트리밍으로 첫 토큰 ~500ms

AS-IS의 단일 LLM 처리를 TO-BE의 의도 분류 → 필요한 요청만 검색·생성 흐름으로 바꾼 라우팅 구조입니다.
불필요한 LLM 호출을 줄여 비용 감소

COMPARE

컨텍스트 엔지니어링 - 로컬 임베딩 + 메모리 시스템 + SSE 스트리밍

AS-IS

외부 의존성: OpenAI Embedding API를 쓰면 요청마다 비용이 발생하고 외부 서비스에 의존해야 했습니다.

UX 지연: 응답 완료까지 3-5초가 걸리면 사용자가 기다리기 어렵고, 대화 흐름이 끊길 위험이 컸습니다.

문맥 단절: 세션 문맥이 유지되지 않아 같은 질문을 반복해야 했고, 어떤 정보를 언제 보여줄지에 대한 설계도 부족했습니다.

TO-BE

로컬 임베딩: ChromaDB + all-MiniLM-L6-v2로 API 비용을 0으로 만들고 외부 의존성을 제거했습니다.

문맥 유지: 최근 10턴 대화를 SQLite에 저장하는 단기 메모리 시스템으로 세션 생성·조화·삭제까지 관리했습니다.

응답 체감 개선: SSE 토큰 스트리밍으로 첫 토큰 ~500ms, 200건 단위 배치 임베딩과 재임베딩 스킵으로 비용도 함께 최적화했습니다.

요청당 비용

AS-IS TO-BE
~\$0.02 → **~\$0.003**

GPT-4o-mini 토큰 단가 기준 추정치로, 불필요한 LLM 호출을 줄여 약 85% 절감했습니다.

첫 토큰 응답 시간

AS-IS TO-BE
3-5s → **~500ms**

SSE 스트리밍으로 응답 완료를 기다리지 않고 첫 토큰부터 바로 보여주도록 바꿨습니다.

세션 문맥 유지

AS-IS TO-BE
0턴 → **10턴**

최근 10턴 대화를 저장해 세션 내 문맥을 유지하고 반복 질문을 줄였습니다.

회고

핵심은 챗봇을 더 화려하게 만드는 것이 아니라 언제 LLM을 호출하지 않을지를 먼저 결정하는 것이었습니다. 의도 분류 노드 하나로 비용이 85% 줄었고, 이 경험이 멀티 에이전트 라우팅 설계 판단으로 이어졌습니다.

의도 분류를 경량 분류 모델로 교체하고 자체 평가셋을 붙여 latency와 비용을 더 낮출 계획입니다.

CONTRIBUTION

검색 품질, 사용자 의도, 서빙 안정성, 비용 효율을 각각 따로 본 것이 아니라 하나의 서비스 흐름으로 다뤄왔습니다. 당근처럼 로컬 맥락이 강하고 빠르고 정확한 탐색 경험이 중요한 서비스에서는, 모델 성능 숫자만이 아니라 실제 탐색 경험과 운영 가능성까지 함께 개선하는 접근이 중요하다고 생각합니다.

당근 ML팀에서 이렇게 기여하겠습니다

01 중고거래 검색 랭킹 - 사용자 의도에 맞는 매물이 먼저 보이게

지역 기반 중고거래 서비스의 검색에서는 짧은 키워드 하나에 구매 의도, 정보 탐색, 특정 카테고리 탐색이 함께 섞일 수 있습니다. 특히 다양한 로컬 도메인을 다루는 검색일수록 의도 해석과 랭킹 조정이 함께 가야 한다고 생각합니다. 저는 이런 문제를 **의도 분류 → 후보 생성 보정 → 랭킹 조정**의 단계로 나눠 접근하는 방식에 익숙합니다. PyMate에서는 쿼리 정제 → BM25+시맨틱 융합 → Reranker로 관련 문서 상위 노출을 만들었고, Seoul Culture Map에서는 의도 분류 노드 하나로 불필요한 LLM 호출을 85% 줄였습니다. 검색 품질을 **의도 파악 → 후보 생성 → 랭킹 → 비용 통제**까지 하나의 흐름으로 다뤄본 경험이 있습니다.

02 콘텐츠 품질 관리 경량 필터 + LLM은 경계 케이스만

동네생활에서 광고성 게시글이나 부적절 콘텐츠를 필터링할 때, 단순 키워드 필터는 우회가 쉽고 LLM 전수 검사는 비용이 과합니다. WorkFlow Agent에서 설계한 **4중 Guardrail(키워드 매칭 → 존재 검증 → 카테고리 제한 → 일관성 모니터링)** 구조를 적용하면, 명확한 위반은 경량 필터로 빠르게 차단하고 LLM은 모호한 경계 케이스에만 집중시켜 비용을 통제하면서 precision을 유지할 수 있습니다.

03 ML 모델 배포 후 품질 판단 숫자 해석 + 모니터링 설계

'정확도 85%'라는 숫자만으로는 서비스 품질을 판단할 수 없습니다. 대규모 검색·추천 환경일수록 품질 개선은 빠르고 안정적이며 비용 효율적인 운영과 함께 가야 한다고 생각합니다. 어떤 케이스에서 틀리는지, 틀렸을 때 사용자 경험에 미치는 영향은 얼마인지, 모니터링 지표는 무엇인지를 함께 설계해야 합니다. LoRA 실험에서 328건 평가셋의 결과를 해석할 때 신뢰구간을 고려했고, RAGAS에서는 4개 지표를 분리해 "어디서 틀리는가"를 특정한 뒤 다음 실험을 설계했습니다. 오프라인 지표를 해석하고 제품 개선 액션으로 연결하는 것이 제 역할입니다.

**당근 ML팀에서
측정 가능한 품질 개선을 배우고 만들고 싶습니다.**

모르는 상태를 오래 두지 않는 것이 저의 일하는 방식입니다. 병목을 정의하고, 빠르게 실험하고, 숫자로 확인하고, 구조로 고칩니다. 검색 품질 평가·사용자 의도·서빙 안정성·비용 최적화
—이 네 축이 만나는 지점에서, 인턴으로서 당근 ML팀의 문제를 가까이에서 배우며 기여하고 싶습니다.

yge0307@gmail.com · github.com/ykgstar37-lab · yge-portfolio.vercel.app